

IFN647 Week 3 Workshop

Pre-Processing: Stemming and Python Classes

Objectives:

- Continue developing your Python skills in text pre-processing.
 - Learn how to apply stemming as part of the text pre-processing pipeline.
 - Understand how to design and implement custom Python classes to represent documents.
- Compare your solutions from last week with the published code solution.
 - If there are any significant differences, discuss them with your tutor.
 - If you are completely stuck after making a genuine effort, review the provided code solution first and then extend it to complete this week's tasks.

Task 1: Update Task 3 of last week's workshop to print the terms of the document and their frequency in ascending order. Note that dictionaries cannot be sorted, but you can get a representation of a dictionary that is sorted. Try the following commands:

```
>>> x = {1: 2, 3: 4, 4: 3, 2: 1, 0: 0}
>>> {k: v for k, v in sorted(x.items(), key=lambda item:
item[1])}
>>> {k: v for k, v in sorted(x.items(), reverse=False)}
>>> {k: v for k, v in sorted(x.items(), reverse=True)}
```

Examples of output of Task 1:

addition : 1
aires : 1
amid : 1
argentina : 2
argentine : 1
awaiting : 1
axel : 1
bank : 1
between : 1
bocon : 1
bonds : 1
bounce : 2
buenos : 1
bugge : 1
change : 1
congress : 1
deficit : 1
delegation : 1
denominated : 1
dollar : 1
due : 2
during : 1
early : 1
economic : 1
economy : 1
events : 1
expect : 1
expected : 2

```
fernandez : 1
fiscal : 1
foreign : 1
frb : 1
friday : 1
fund : 1
general : 1
government : 1
higher : 1
including : 1
international : 1
large : 1
low : 1
...
```

Task 2: Stemming refers to a crude heuristic process that removes the ends of words in the hope of finding the stemmed common base form correctly most of the time. This process often includes the removal of derivational affixes. Lovins (1968) defines a stemming algorithm as "a procedure to reduce all words with the same stem to a common form, usually by stripping each word of its derivational and inflectional suffixes (and sometimes prefixes)". A popular stemming algorithm is the Porter2 (Snowball) algorithm. You can read the details of Porter2 algorithm in the following link:

<http://snowball.tartarus.org/algorithms/english/stemmer.html>

For Python, please go to

<https://pypi.python.org/pypi/stemming/1.0>

to download Python implementations of porter2 stemming algorithms, follow the instruction to import and use stemmer in your Python code (or see the Canvas).

Use porter2 stemming algorithm to update your last week function `parse_doc(input, stops)` to make sure all terms (words) are stemmed. Then display document's stems and their frequencies in ascending order as you did in Task 1. You can compare the results of Task 1 and Task 2 to analyze the difference in the extracted terms (i.e., words versus stems).

Task 3: Define a `Doc_Node` Class:

```
class Doc_Node:
    def __init__(self, data, next=None):
        self.data=data
        self.next=next
```

where "data" attribute stores the document's information, i.e., tuple (docid, curr_doc), and

- `docid` is the 'itemid' in <newsitem>
- `curr_doc` is a dictionary of term_frequency pairs (review last week workshop if you do not understand the tuple).

You are also required to define a linked list class:

```
class List_Docs:
    def __init__(self, hnode):
        self.head=hnode
```

And its methods:

```
insert(self, nnode)
```

```
# append nnode to the end of the linked list
lprint(self)
# print out the linked list
```

Task 4: Design a main function to read a set of xml files and represent each file as a node, then create a linked list to link all nodes together. You need to update function **parse_doc()**, such as arguments or return value, and then use **Doc_Node** and **List_Docs** classes.

Examples of output

(ID-6146: 72 terms) --> (ID-741299: 96 terms) --> ...